GCS SCENARIO BASED Q&A

BY - SHUBHAM WADEKAR

© Shubham Wa

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

1. Your company wants to separate raw, curated, and analytics data zones in GCS. How would you design the bucket/folder structure?

Start with the goals up front: immutability for raw, discoverability and schema stability for curated, and fast query/layout for analytics. Those goals drive the namespace and access model - raw needs write-once, append-only paths; curated needs partitioned columnar layouts and metastore registration; analytics needs formats and locations optimized for the query engine (BigQuery, Presto, etc.). Keep the structure predictable so automation and manual consumers can find data without guessing.

Choose either a small number of well-structured buckets or separate buckets per zone depending on security and operational constraints. A common, pragmatic pattern is one bucket per environment (e.g., gs://myproj-prod-data) with three top-level prefixes: /raw/, /curated/, /analytics/. Example layout:
gs://myproj-prod-data/raw/<source>/<yyyy>/<mm>/<dd>/
gs://myproj-prod-data/curated/<domain>/<table>/partition=<yyyy-mm-dd>/
gs://myproj-prod-data/analytics/<dataset>/<table>/date=<yyyy-mm-dd>/
This keeps cross-zone operations simple and lets you apply uniform lifecycle policies while still enforcing access via IAM conditions on prefixes when needed.

If regulatory or tenant isolation requires stricter separation, use separate buckets per zone (e.g., gs://myproj-prod-raw, gs://myproj-prod-curated, gs://myproj-prod-analytics). That makes lifecycle, encryption (CMEK), and VPC-SC boundaries simpler and reduces the blast radius for accidental privileged access. The trade-off is more administrative overhead and slightly more complexity for cross-zone jobs that copy or transform data.

Inside each zone follow concrete conventions: enforce partitioning by date/business key, avoid many tiny files (compact during ETL), write _SUCCESS + manifest files on job completion, and include metadata JSON (job id, schema version, source files) alongside outputs so replays are deterministic. Register curated tables in a metastore (Dataproc Metastore or Hive Metastore) and Data Catalog with schema/version, and store schema evolution records in a migrations repo so consumers can adapt safely.

Finally, implement governance: lifecycle policies (move raw to cold storage after X days, auto-delete temp folders), distinct IAM roles (producers can write to raw/ but not curated/), logging/audit sinks, CMEK if required, and cost tags (labels) on objects or buckets. Automate validation (row counts, checksums) and add an orchestration step that atomically promotes a curated partition only after checks pass - this keeps downstream analytics stable and reliable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

2. A project requires storing data for users across multiple regions. How would you decide between regional and multi-regional buckets?

Make the decision from three primary axes: compliance/residency, latency & performance, and cost/operational complexity. If regulation or contracts require data to remain inside a particular country/region, pick a regional bucket in that specific location - compliance trumps everything. Regional buckets give you clear geographic residency and are colocated with regional compute to avoid cross-region egress and offer predictable performance for local users.

If you need the highest global availability and simpler management for globally distributed reads (no strict residency requirement), multi-regional (or dual-region) buckets reduce the complexity of replicating data yourself and provide automatic geographic redundancy. Multi-regional storage is convenient when many consumers across regions need read access and you want the provider to handle replication and durability for you. Consider, however, that multiregional placement can affect write/read latency depending on where your compute lives, and pricing/reservation models may differ.

Also consider where processing happens. Co-locate storage with compute: if your analytics/ETL runs mostly in one region (BigQuery dataset, Dataproc cluster), use a regional bucket in that region to avoid egress charges and to get better throughput. For write-heavy ingestion spread across multiple regions, you can ingest into regional buckets nearest the producers and asynchronously replicate/aggregate into a central multi-regional or analytics-focused bucket - this hybrid approach balances locality and central analytics needs.

Factor in operational controls and cost: regional buckets let you apply tighter network/VPC restrictions and reduce unexpected egress when compute runs nearby; multi-regional simplifies availability but may cost more and make it harder to meet strict residency rules. Also evaluate read patterns - if most reads are global but latency tolerance is high, multi-regional is simpler. If reads need low latency per region, consider per-region buckets plus a replication/sync strategy or CDN layer for hot objects.

In practice, pick rules you can automate and document: if compliance == true -> regional in required jurisdiction; else if global-read and low ops overhead desired -> multi-regional; else (mixed workloads) -> regional buckets near producers/consumers with a central analytics bucket and managed replication pipelines. Whichever you choose, codify it in your infra templates, tag buckets with location/purpose, and validate the design using a small pilot measuring latency, throughput, and egress/cost impact before rolling out broadly.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

3. You need to batch-ingest daily flat files from an on-prem server into GCS. How would you design the pipeline?

I'd keep it simple and reliable: a daily pull or push from on-prem into a "raw" GCS path, wrapped with manifesting, validation, and idempotent promotion. On the on-prem side, I'd schedule an extraction job that lands flat files (CSV/TSV/JSON) into a staging folder with stable, timestamped names and a daily manifest listing expected files plus checksums (MD5/SHA256). That manifest becomes the contract for completeness and integrity.

For transfer, I'd use Google Storage Transfer Service (STS) with the on-prem agent, or SFTP-to-GCS via a small transfer runner if STS isn't an option. STS gives me resumable, parallelized copies, checksum verification, and retry out of the box. I'd schedule it to pull the previous day's folder over a private link (VPN/Interconnect) and land into gs://bucket/raw/<source>/ingest_date=YYYY-MM-DD/. If security needs encryption-in-flight and at-rest, I'd enable TLS and CMEK on the bucket; if the files are sensitive, consider PGP at source and decrypt in cloud after integrity checks.

Once the files arrive, an orchestration step (Cloud Composer/Airflow or Workflows) validates the drop: compare manifest vs objects in GCS, verify checksums, and quarantine any mismatches to an error path. I'd make the pipeline idempotent by writing to a temporary .../_staging/ prefix first and only "promoting" (move/rename via copy+delete) to the final partition after the checks pass. Promotion writes a small _SUCCESS marker so downstream jobs know the partition is safe to read.

To avoid duplicates and partial re-runs, I'd key each file by (source_file_name, business_date, checksum) and keep a tiny state table (BigQuery or Firestore) recording what was accepted. If a rerun happens, the pipeline sees the key already processed and skips or overwrites safely. I'd also normalize the files quickly: optional lightweight Dataproc/Dataflow step to convert CSV → compressed Parquet, enforce schemas, and compact tiny files so downstream reads are efficient.

Finally, I'd add ops hygiene: alerts for late or missing files, per-day row counts vs historical baselines, and cost tags. Access is least-privilege: on-prem service account can only write to the raw prefix; only ETL can read raw and write curated; analysts never touch raw. With this, you get a predictable, auditable daily feed that's easy to support and cheap to run.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

4. Streaming IoT sensor data must land in GCS for further ETL. How would you implement this using Pub/Sub and Dataflow?

I'd start at ingestion with Pub/Sub as the durable buffer. Each device (or edge gateway) publishes events to a topic with attributes like device_id, event_time, and site. If devices speak MQTT, I'd place a thin gateway (Cloud Run or a managed bridge) that authenticates devices, validates payloads, and forwards to Pub/Sub; otherwise, direct HTTPS publishing with service credentials works. Topics are partitioned by domain (e.g., telemetry, alerts) to keep schemas clean and consumers decoupled.

Dataflow then does the heavy lifting from Pub/Sub to GCS. I'd build a streaming pipeline that reads from Pub/Sub, parses/validates messages, and assigns event time from the payload. Use fixed or session windows (e.g., 1–5 minutes) with triggers to roll files, and allow late data with a reasonable lateness bound (e.g., 30–60 minutes) so out-of-order readings still land in the right partitions. Bad or unparseable messages go to a dead-letter Pub/Sub topic with enough metadata to debug.

For file outputs, I'd write compressed Parquet (or Avro) to GCS with deterministic partitioning like gs://bucket/raw/iot/ingest_date=YYYY-MM-DD/hour=HH/device_date=YYYY-MM-DD/ and shard files by window to avoid tiny objects. Dataflow's file sinks handle windowed rolling nicely; I'd also enable temporary staging paths and finalize/rename semantics so readers don't pick up partial files. If exactly-once is important at file level, I'd include a content-hash in the file name or write a manifest per window with checksums to detect duplicates.

Operationally, I'd lean on Dataflow's autoscaling and backpressure handling. Set sensible max workers for bursts, monitor system and custom metrics (lag, late data ratio, DLQ rate), and scale topics/partitions if QPS grows. IAM stays tight: publishers can only write to their topic; the pipeline's service account has Pub/Sub read and GCS write on specific prefixes. Buckets use CMEK where required, and VPC-SC if the environment is locked down.

Finally, I'd close the loop with quality and evolution. Schemas live in a contract (Avro/Proto/JSON schema) with additive evolution only; validation happens in the pipeline and violations route to DLQ with alerts. Downstream ETL (Dataproc or another Dataflow job) reads these Parquet files on a cadence, compacts them if needed, and promotes into curated tables. This design keeps ingestion resilient, storage efficient, and downstream processing straightforward.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

5. You have CSV files in GCS that must be loaded into a partitioned BigQuery table. How would you orchestrate this?

I'd treat this as a reliable, idempotent ETL DAG and use Airflow / Cloud Composer (or Workflows if you prefer a lighter orchestration) as the orchestrator. The DAG starts with a sensor that watches for a daily manifest or GCS prefix (using GCS notifications → Pub/Sub or an Airflow GCSPrefixSensor) so you only trigger when the expected files for a partition arrive. Using a manifest (list + checksums) makes completeness and retry logic much simpler than relying on "any object appears".

The next step in the DAG is validation and normalization: run a lightweight job (Cloud Function or Dataflow / small Dataproc job) that checks CSV schema/headers, enforces field types, normalizes timestamps, and writes a small normalized copy (or schema-compliant Parquet) to a staging prefix. Don't rely on autodetect in BigQuery for production - validate schema in code and fail fast on drift so you don't silently load bad data. Also produce a _SUCCESS marker once staging is correct.

Then submit a BigQuery load job that targets the partitioned table. Use load jobs (not streaming inserts) for cost and throughput: point sourceUris at the staging files in GCS, set the correct CSV options (skipLeadingRows, fieldDelimiter, allowJaggedRows only if you must), provide the explicit schema, set writeDisposition to WRITE_APPEND (or write to a staging table and then MERGE into the partitioned production table for stronger idempotency), and set the partition decorator (or load into the partitioned table using a DATE/TIMESTAMP column). For very large daily volumes prefer compressed newline-delimited CSV → converted to Parquet first, then use Parquet loads for best performance and lower bytes scanned.

Make the flow idempotent and safe: either (a) load into a staging BigQuery table named by run (or partition) and run a deterministic MERGE into the target partition, or (b) if loading directly to partition, enforce a unique load-id column and use INSERT/MERGE semantics to avoid duplicate rows on retry. Record each logical ingest event in a small control table (BigQuery or Firestore) with (source_manifest, checksum, ingest_time, status) so the orchestrator can skip re-processing already-successful manifests.

Finally add monitoring and governance: emit metrics (rows loaded, bytes, load duration), set alerts on load failures or row-count deltas, store logs in Cloud Logging, and use IAM so only the ETL service account can write to the production dataset. Add lifecycle rules (partition expiration, clustering on high-cardinality query keys) and automated tests (row counts, checksum comparisons) at the end of the DAG so downstream consumers can trust the partitioned table.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

6. Analysts want to query data directly from GCS without ingestion. How would you set up federated queries?

I'd expose the GCS objects as BigQuery external (federated) tables so analysts can query them with SQL without copying data into BigQuery storage. Create external table definitions that point to your GCS URIs (wildcards allowed) and specify the correct format (CSV, Parquet, Avro, ORC, or newline-delimited JSON). For production use, prefer columnar formats (Parquet/Avro) over CSV for much better performance and lower query costs.

Don't rely on schema autodetect for long-term use - explicitly define schemas for external tables (or keep schema files in Avro/Parquet metadata). If your GCS data is partitioned via a Hive-style directory layout (e.g., date=YYYY-MM-DD/), enable Hive partitioning options when you create the external table so BigQuery can prune partitions and avoid scanning unnecessary files. That makes federated queries much more efficient and cost-effective.

Control access and governance: grant analysts bigquery.dataViewer (or finer roles) on the external dataset and restrict direct GCS access - keep GCS permissions limited to ETL service accounts. If analysts shouldn't see raw columns, create authorized views or materialized views in BigQuery that present a curated schema over the external table. Also add Data Catalog entries for discoverability and document schema/versioning so users know what they're querying.

Be explicit about performance trade-offs: external tables scan data in place and may be slower or costlier for heavy, repeated analytical queries than native BigQuery tables. For frequently-run, high-concurrency queries create scheduled materialized tables or run periodic loads into native partitioned & clustered tables. Use query caching, cached results, or scheduled extracts for repetitive workloads to control cost and improve latency.

Finally, operationalize the pattern: automate external table creation in IaC, keep schema evolution policies (how to add columns, when to migrate to native tables), and monitor query bytes and latencies so you can decide when to promote an external dataset into BigQuery storage. Include tests that validate partition discovery and sample-query results so analysts can rely on federated data for ad-hoc analysis safely.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. You must read JSON files from GCS, transform them, and write the output back as Parquet. How would you design this Dataflow pipeline?

Start by treating this as a batch Beam job: use FileIO.match / FileIO.readMatches (or TextIO if files are newline-delimited JSON) to read the JSON objects from the GCS prefix. Keep input discovery robust by using manifests or stable prefixes and by validating file completeness before kicking off the pipeline so you don't accidentally process partial uploads. If schema is known, capture it up front; if it can change, include a lightweight schema discovery step and fail-fast on incompatible records.

Next, parse and validate each JSON record as the first transform. Use a well-tested JSON parsing library in your chosen SDK (Java/Python) and convert into a typed Beam PCollection<Row> or an Avro/POJO object so downstream transforms are structured. Add validation that enforces types, required fields, and timestamp normalization; route bad rows to a dead-letter sink (GCS or Pub/Sub) with the original payload plus an error reason to make debugging straightforward.

Then implement the transformation logic in small, composable ParDos or Map transforms - enrichments, lookups (side inputs from small reference tables in GCS or Cloud Storage), masking, and column renames. Keep transforms pure and idempotent so replays are safe. If you need joins with large reference datasets, either stage them as side inputs if small or perform them as keyed GroupByKey operations or use a two-step lookup (e.g., write intermediate keyed files and join with Dataproc if massive).

For the Parquet write, use Beam's file sinks (Avro/Parquet IO where available) and write to a staging/temp directory first. Configure sensible sharding (number of shards) and file-size targets so Parquet files are large enough for efficient downstream reads (aim for tens to hundreds of MBs). Ensure compression and Parquet row-group sizes are tuned, and include a _SUCCESS marker or write a manifest that lists produced files and their checksums to signal job completion atomically to downstream consumers.

Finally, operationalize: run the job as a Dataflow batch or template, enable autoscaling, point staging/temp and output to the correct GCS buckets with proper IAM and CMEK if needed, and export metrics (records processed, dead-letter counts, file sizes). Add end-to-end tests (small sample input → expected Parquet schema + row counts), and CI/CD to build and deploy the template. If latency requirements are low, this batch Dataflow pattern is simple and reliable; if near-real-time is required, convert to a streaming pipeline with windowing and exactly-once sink considerations.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

8. A streaming Dataflow pipeline is writing too many small files to GCS. How would you optimize this?

First principle: small files usually mean too-frequent finalization. Move from per-record or very-small-window writes to windowed, coalesced writes. Use Beam's windowing and FileIO.write() (or WriteToFiles/Parquet sink) with reasonable window durations (e.g., 1–5 minutes or larger depending on latency tolerance) so each window produces a handful of larger files instead of thousands of tiny ones. Increase the allowed lateness carefully so late events still get placed correctly without exploding file counts.

Second, control sharding and file size targets explicitly. Configure the sink to use fewer shards per window (or a target bytes-per-file) so each output file reaches an efficient size (tens to hundreds of MB). Avoid forcing a single shard unless your throughput is low - one shard can become a bottleneck. If your Beam IO supports it, use withNumShards() or withMaxRecordsPerFile() / withMaxBytesPerFile() knobs to let the runner create compact files.

Third, buffer and batch within the runner instead of immediately flushing to GCS. For example, use GroupIntoBatches or accumulate records keyed by window and then write once per bundle. If you need exactly-once guarantees, use the sink's transactional/atomic finalize features (write to temp + finalize/rename) rather than many tiny commits. Avoid using GCS compose as a primary strategy (it's limited and can add complexity) - better to produce correctly-sized files from Dataflow directly.

Fourth, if streaming constraints require very low latency for some consumers but you still want compact files for analytics, split responsibilities: have a low-latency small-file stream (or Pub/Sub topic) for real-time consumers, and a downstream compaction job (periodic Dataflow or a Dataproc Spark compaction job) that merges small files into large Parquet files for analytics. This hybrid approach preserves low-latency needs while keeping long-term storage efficient.

Finally, monitor and iterate: emit metrics for output file count, average file size, and write latency; set alerts for excessive small-file rates. Run load tests to find the sweet spot for your windows, shard counts and row-group sizes, and document the trade-offs so downstream teams understand latency vs storage efficiency. With tuned windowing, controlled sharding and occasional compaction, small-file problems disappear while keeping pipeline semantics intact.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

9. You are running Spark jobs on Dataproc. Would you use HDFS or GCS for storage? Why?

I'd use GCS as the primary storage for most real-world Dataproc workloads. GCS is durable, highly available, and decouples storage lifetime from ephemeral clusters - you can destroy and recreate Dataproc clusters without losing data. That separation simplifies operations, lowers cost (no need for always-on HDFS masters), and lets multiple engines (Spark, Presto/Trino, Dataflow, BigQuery external reads) share the same data without copying.

GCS also gives simpler security and governance: IAM, uniform lifecycle rules (nearline/coldline), CMEK, and audit logging are easier to manage than running a self-managed HDFS with Kerberos. For cloud-native patterns you get better integration (Cloud Monitoring, Transfer Service, Storage API) and generally predictable throughput when co-located with compute in the same region - which avoids cross-region egress costs and keeps latency acceptable for batch ETL.

That said, I still use HDFS-style local storage on the cluster for transient, shuffle-heavy operations and very small temporary files. Spark's shuffle and some legacy libraries perform best when using local disk (ephemeral SSDs) for intermediate spill; Dataproc configures appropriate local disks for that. Also, if you have strict low-latency small-file workloads that require POSIX semantics, an HDFS-like layer can help - but try to keep those local to the cluster or use a table format that hides the complexity.

If you need transactional updates, ACID semantics or incremental ingestion patterns, I prefer using table formats (Hudi/Iceberg/Delta) on top of GCS rather than a persistent HDFS cluster. These formats provide snapshotting, upserts and compaction while keeping storage on GCS so you retain the cloud benefits and enable schema evolution and efficient reads by downstream engines.

In short: default to GCS for persistent storage and sharing; use local HDFS-like storage only for ephemeral, performance-sensitive intermediates. Architect your jobs to read/write durable artifacts to GCS and rely on cluster-local disk for shuffle and temporary spills - that pattern gives the best mix of durability, cost-efficiency, and performance on Dataproc.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

10. Your Spark job performance is poor because of thousands of small files in GCS. How would you fix it?

First, stop the root cause: change upstream producers or ETL to write larger files - batch writes, increase flush/rollover thresholds, and avoid per-record or tiny-window commits. If the producer is a streaming sink, increase window size or use batching so each sink write produces fewer, bigger files. Preventing tiny files is far cheaper and more effective than constantly compacting later.

Second, run a compaction job as part of your ETL: use a Dataproc Spark job that reads the small files for a partition, coalesce(n) or repartition(n) to a sensible number of output files, and write back as compressed Parquet with large row-groups (target tens to hundreds of MB per file). Schedule compaction regularly (daily/weekly) after ingestion so downstream queries and future jobs read efficient, large files instead of millions of tiny objects.

Third, adopt a table format that supports file lifecycle and incremental compaction: Hudi, Iceberg or Delta can manage small-file churn by committing new files and running compaction/cleaner operations automatically. They also make incremental reads faster and support upserts which often reduce small-file proliferation in the first place. These formats work well with Dataproc and keep data on GCS while providing operational primitives to fix small-file issues.

Fourth, tune Spark and IO settings while you transition: increase spark.sql.shuffle.partitions appropriately (not too high), avoid creating many tiny output partitions, and set spark.sql.files.maxPartitionBytes / spark.sql.files.openCostInBytes so Spark creates fewer partitions when reading many small files. Also consider enabling vectorized Parquet readers and tuning GC/serializer settings to reduce per-file overhead during read.

Finally, add monitoring and prevention: emit metrics for file count per partition, average file size, and compaction backlog; alert when tiny-file ratios exceed thresholds. Automate compaction, enforce producer-side batching via contracts/manifests, and run periodic audits so the environment doesn't slip back into the small-files problem. With prevention + compaction + format-level tools, you'll regain predictable performance and lower cost.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. You need a DAG to trigger whenever a new file lands in GCS. How would you configure it?

I'd avoid polling sensors and use event-driven triggers: enable GCS object-change notifications that publish to a Pub/Sub topic whenever an object is finalized, and subscribe with a small Cloud Function (or Cloud Run service) that receives the Pub/Sub event. The Cloud Function's job is lightweight: validate the event (bucket, prefix, metageneration/finalized state), optionally check file stability (size unchanged after a short wait), and then trigger the Composer/Airflow DAG with the object path as a parameter via the Airflow REST API or by publishing a trigger message to a DAG-specific Pub/Sub topic that an in-DAG sensor pulls. This keeps latency low and avoids expensive, long-running Airflow sensors.

Make the trigger idempotent and secure: embed a unique event id or object checksum in the DAG run payload so repeated Pub/Sub deliveries don't cause duplicate work. Authenticate the Cloud Function call with a least-privilege service account that's allowed to trigger the DAG (use IAM or Airflow's REST auth). Also implement a dead-letter path for malformed or unauthorized events so you can triage without flooding upstream systems.

Inside Composer, design the DAG to accept the GCS path as a runtime parameter and immediately perform lightweight validation (manifest presence, checksum, file age). If the file is expected as part of a batch, compare against a manifest or control table to ensure completeness before starting heavy ETL. Keep the first DAG task cheap so you can bail early for late/partial uploads and avoid wasting compute on ephemeral Dataproc clusters.

For reliability, add retries with exponential backoff in the Cloud Function and DAG tasks, and surface failures to an alerting channel. Use structured logging (include object path, run id, size, event timestamp) so you can trace a file from arrival through processing. Also add monitoring on Pub/Sub subscription delivery failures and DAG trigger rates so you detect spikes or backpressure early.

Finally, document the contract (what events will trigger, required metadata, prefix rules) and provide a small test harness for producers to validate they can trigger the DAG correctly. If you expect very high ingest rates, consider sharding topics by prefix or batching trigger events into aggregate jobs to avoid overwhelming Composer with too many concurrent DAG runs. This event-driven pattern is fast, cost-efficient and avoids the waste of continuous Airflow polling.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

12. After ETL, processed files must be moved to an archive bucket. How would you design the workflow in Composer?

I'd make the archive move an explicit, final task in the ETL DAG that only runs on successful completion of all upstream tasks - that keeps semantics simple (success → archive, failure → keep for triage). The task should copy objects from the staging/raw prefix to the archive bucket using the GoogleCloudStorageToCloudStorageOperator (or a small Python gcs client task) and then verify the copy by checksum or object size before deleting the source. Do copy-then-verify-then-delete rather than a blind rename to avoid accidental data loss.

Keep the archive layout predictable and metadata-rich: write to gs://archive-bucket/<source>/date=YYYY-MM-DD/ and include a metadata JSON alongside each file (ingest run id, schema version, original object name, checksum). This helps future restores and audits. Also add object labels or bucket-level labels for billing and ownership so archived data is easy to attribute later.

Handle failure modes: if the copy or verification fails, leave the original files in a quarantine/staging folder and push a structured alert with run-id and file list. Include a small retry policy (exponential backoff) for transient GCS errors, and cap retries so the DAG can fail fast and open a ticket. For repeated failures, optionally create a snapshot or keep a debug copy before deleting so you can investigate without data loss.

For idempotency and safety, record the archive operation in a control table (BigQuery or Firestore) with (file_path, checksum, archive_path, status, archived_at). The DAG should consult this table and skip re-archiving already-archived files. This prevents duplicate work on re-runs and makes rollbacks straightforward (you can detect which files were moved by run id).

Finally, consider lifecycle automation: if archive is long-term cold storage, set bucket lifecycle rules (nearline/coldline/class A/B) and retention policies instead of manual moves for older data. But keep the Composer archive step as the authoritative bookkeeping action because it lets you add metadata, trigger downstream retention policies, and guarantee that ETL provenance is captured before data leaves the active zone.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

13. A downstream ML pipeline requires columnar storage. Would you choose Parquet or Avro, and why?

Parquet - hands down - for ML feature pipelines. Parquet is a true columnar storage format: it stores each column separately, supports column-level compression and encoding (dictionary, run-length), and exposes column statistics and row-group metadata that query engines and Spark can use to skip work. For ML you usually select a subset of columns (features) for training or scoring, and Parquet lets you read only those columns much faster and with far less I/O than a row format.

Vectorized/read-optimized engines (Spark, Arrow-based readers, BigQuery external reads) get big wins with Parquet: faster deserialization, predicate and projection pushdown, and better memory locality - all of which reduce training/feature-extraction time and cost. Parquet also supports efficient nested types (structs, arrays) with good performance characteristics, so you don't have to denormalize everything just to get columnar benefits.

Avro's place is different: it's a row-oriented, write-friendly format with excellent schema evolution semantics and small write/append costs. Avro is great for raw ingestion, message payloads, and when you need frequent small writes or streaming. But for downstream ML workloads that read large datasets and scan columns, Avro will be slower and more I/O-heavy because it forces reading whole records to extract fields.

A practical pattern I often recommend in interviews: ingest events as Avro/JSON (for flexible, append-heavy writes), validate and stage raw data, then run a periodic transform to convert/compact into Parquet for curated and analytics layers. That gives you the best of both worlds - simple ingestion + fast, columnar reads for ML. Also mention codec choices: use Snappy or Zstd for Parquet (Snappy for lower CPU/fast reads; Zstd if you want better compression at the cost of CPU).

Finally, tune Parquet for ML: pick sensible row-group and file-size targets (tens–hundreds of MB), partition by date/tenant for pruning, and use clustering or column statistics where supported. Register tables in a metastore or Data Catalog so downstream workloads can discover schema and partitions. Overall: Parquet for ML consumption, Avro for raw/ingest - convert and compact into Parquet before heavy analytics or model training.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

14. You are ingesting logs in JSON but queries are slow. How would you improve performance with file format and compression?

First, stop querying raw JSON blobs directly. JSON is convenient for ingestion but expensive to parse and scan repeatedly. Convert the logs to a columnar format (Parquet or ORC) as early in the pipeline as possible - ideally in the ETL stage - so queries can read only needed columns, benefit from predicate pushdown, and avoid expensive runtime JSON parsing. If you need schema evolution, stage in Avro or keep a schema registry, then write Parquet for query.

Second, pick the right compression for Parquet. Snappy is the usual default: very fast to compress/decompress and a good tradeoff for query latency. If storage cost matters more and you can afford CPU, Zstd gives much better compression ratios (lower I/O) which can speed scans when I/O is the bottleneck. Avoid codecs that prevent splitting or increase read latency (e.g., gzip for large files) - prefer splittable, columnar-friendly codecs.

Third, partition and cluster your data so queries touch minimal files. Partition by ingestion date (and possibly region/tenant) so date-range queries prune partitions. Use clustering (or sort/partition by hot filter columns) to reduce scanned data within partitions. Also ensure you compact tiny JSON files into fewer, larger Parquet files (target tens-to-hundreds of MB) because thousands of small files dramatically slow metadata handling and increase per-file overhead.

Fourth, flatten frequently-queried fields into top-level columns during ETL rather than leaving them as nested JSON that must be extracted at query time. Avoid heavy use of runtime JSON_EXTRACT functions in SQL - they're CPU-intensive and block vectorized readers. Materialize derived columns that analysts commonly use (user_id, status_code, event_type) so queries are simple column scans.

Finally, consider where the data is queried. For ad-hoc or heavy analytical use, load curated Parquet into a warehouse (BigQuery native tables) rather than relying on external/federated queries over GCS. BigQuery will offer partition pruning, clustering, and faster execution. Add monitoring (bytes scanned, average file size, query latencies) and iterate: choose codecs, partitioning, row-group sizes and compaction cadence based on measured bottlenecks rather than guesswork.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

15. You must store daily transactions in GCS for BigQuery ingestion. How would you design partitioned directories?

I'd use a clear, consistent, Hive-style partition layout keyed by the event or business date that BigQuery will use as the partition column - for example gs://bucket/raw/transactions/source=<src>/year=YYYY/month=MM/day=DD/. This makes it trivial for ETL jobs and BigQuery load jobs to target a single day with a simple wildcard, and it maps naturally to partitioned tables so partition pruning works end-to-end. Keep the partition key based on event time (or business date) rather than ingestion time unless you have a strong reason otherwise.

Decide the granularity upfront: daily is the natural fit for "daily transactions." Only consider hourly if you have very high volume or strict latency SLAs. Also standardize on a timezone (UTC recommended) and document it in the contract, because mismatches between producer and loader timezones are a common source of silent partition drift. Record both the event timestamp inside each record and the partition path so you can detect late-arriving or backfilled data.

Make writes atomic and discoverable: stage files under a _staging/ or tmp/ prefix then move or copy them into the final partition path when the job completes, or write a _SUCCESS marker alongside the partition after validation. Include a manifest file (list of objects + checksums + run-id) per partition so the BigQuery load orchestration can validate completeness and make the ingest idempotent. That avoids partial loads and makes retries safe.

Tune file layout and size: write compressed columnar files (Parquet) where possible and target file sizes in the tens-to-hundreds of MB range to avoid many small objects. Name files with deterministic patterns that include run id and shard (e.g., part-0000-run-<id>.parquet) so debugging and replay is easy. Also add a small metadata JSON in the partition with schema version and job metadata so downstream consumers know what produced that partition.

Add governance and automation: register the partitioned dataset in a metastore or Data Catalog, enforce IAM so only ETL service accounts can write to raw partitions, and schedule lifecycle rules (move older raw to cheaper storage or set expiration) if required. Finally, make your BigQuery load process either load from the partition path directly (using partition decorators or wildcard URIs) or load into a staging table and MERGE into production to get strong idempotency and easy rollbacks.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

16 Your Spark job is reading too much irrelevant data from GCS. How would partitioning improve performance?

Partitioning lets your job read only the data that matches the query predicates instead of scanning the entire dataset - that's the core win. If your queries filter on date, customer_id, region, or another stable key, store data using that key in your folder layout (Hive-style key=value/) so Spark (and other engines) can prune partitions at read time and skip remote fetches and metadata operations for unrelated files. That reduces both latency and egress/cost dramatically.

Choose partition keys that align with your most common filters. For transactional data, date partitions (year/month/day) are the most common and effective. If users frequently filter by tenant or region as well, consider a two-level partitioning (date=.../region=...) or partition by date and use clustering on high-cardinality columns (customer_id, product_id) to get additional pruning without exploding the number of partitions.

Avoid over-partitioning: too many tiny partitions creates metadata overhead and small-file problems. Aim for a balance - partitions that cover useful pruning ranges but not so fine-grained that each partition is only a few kilobytes. If a natural key would produce many partitions, prefer partitioning at a coarser level and use clustering or bucketing for the high-cardinality dimension instead.

Make sure Spark and your metastore know the partition scheme (register tables in Hive Metastore / Dataproc Metastore or use Spark's spark.read.option("basePath", ...) with Hive-style dirs) so partition pruning happens automatically. Also tune Spark's file-read settings (spark.sql.files.maxPartitionBytes, spark.sql.shuffle.partitions) to avoid creating too many small input splits when reading many files inside a partition.

Finally, operationalize and measure: instrument bytes read per job, partition prune ratios, and query latencies so you can validate the benefit and iterate. If you still see high read volumes, add compaction jobs to merge tiny files inside partitions and consider promoting frequently-queried partitions into BigQuery (or cached Parquet snapshots) for even better performance. Partitioning plus compaction and good matching of partition keys to query patterns will usually remove the "reading too much irrelevant data" problem.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

17. You need to design a GCS data lake with raw, curated, and analytics zones. How would you implement it?

I'd start by creating a clear physical and logical separation: either one environment bucket with top-level prefixes (/raw/, /curated/, /analytics/) or three buckets (proj-raw, proj-curated, proj-analytics) if you need stricter IAM, CMEK, or VPC-SC boundaries. Use the simpler single-bucket layout when policies are uniform and the multi-bucket approach when you must enforce isolation, different retention rules, or billing segmentation. Name and label everything consistently so automation and people can find data without guessing.

In the raw zone I'd accept immutable, append-only drops from producers. Keep files in producer-native formats and include manifests and a _SUCCESS marker for completeness. Store raw under Hive-style, time-partitioned prefixes like raw/<source>/year=YYYY/month=MM/day=DD/, and require producers to write to a staging prefix and only promote to final when checksums and manifest validation pass. This guarantees deterministic replays and simple provenance.

Curated is where transformation, schema enforcement and compaction happen. Dataproc or Dataflow jobs read raw, validate/enrich, and write columnar, partitioned files (Parquet/ORC) into curated/<domain>/<table>/partition=.../. Register these in a metastore (Dataproc Metastore or Hive Metastore) and Data Catalog so downstream engines see stable schemas and partitions. Use a table-format (Hudi/Iceberg) when you need ACID upserts, time-travel, or compact/cleanup operations - that reduces operational pain later.

Analytics zone is optimized for fast SQL and BI: either BigQuery native tables or curated Parquet external tables that are loaded into BigQuery on a cadence. Keep analytic artifacts denormalized and clustered/partitioned for query patterns, and expose authorized views for analysts rather than giving them direct write access to curated data. Add lifecycle rules (archive to Coldline or delete) and cost labels so analytics storage and compute are trackable.

Finally, bake governance, monitoring and automation into the design: IAM roles scoped per zone (producers → raw only, ETL → curated, analysts → analytics), schema/versioning policies, lineage capture with manifests and job-run metadata, automated tests and promotion steps, and alerts for drift/late files. Use IaC for bucket/prefix creation and orchestration (Airflow/Composer) for ETL so the whole lake is reproducible, auditable and maintainable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

18. Your team wants schema-on-read flexibility in the raw zone but schema enforcement in the curated zone. How would you design this?

Make schema-on-read the explicit contract for raw: accept whatever the producer sends, store the original payload and keep a metadata manifest describing observed fields, producer version, and checksum. Don't mutate raw objects. Provide lightweight validation utilities that consumers can run when they need to interpret raw (for example a schema-discovery job that infers types and records anomalies), but keep raw accessible strictly to ETL/service accounts and auditors - analysts should not write back into raw.

For the curated zone, require a formal schema registry and a promotion process. ETL jobs must validate raw against an expected schema (fail-fast on incompatible or dangerous changes), apply transformations, and emit curated artifacts that strictly conform to the curated schema. Store the curated schema in Dataproc Metastore or Data Catalog, version it, and require CI/CD migration scripts for any breaking changes so schema evolution is controlled and auditable.

Engineer the promotion pipeline to be atomic and idempotent: write curated outputs to a staging prefix, run schema validation and row-count/checksum checks, and only after all checks succeed, register partitions and write a _SUCCESS marker or commit the table-format transaction (Hudi/Iceberg). Keep a small control table that records which raw manifests were consumed to produce each curated partition so you can trace lineage and re-run deterministically if needed.

Handle schema evolution with clear rules: allow additive, nullable column additions with minimal coordination, but require a migration plan for renames, deletions, or type changes. Automate compatibility checks in your CI (sample-run transformations, run consumer queries) and publish change-notices to downstream consumers. For complex evolution, prefer creating a new versioned curated table rather than breaking the existing one, and provide a transition window.

Finally, operationalize monitoring and governance: alert on schema drift detected during raw-to-curated runs, capture statistics on dropped/invalid rows (send to DLQ), and expose discoverability via Data Catalog with owner/contact information. Provide templates and libraries that make writing validation and promotion logic simple for devs so the practice is consistent—flexible raw + strict curated, enforced by automation and clear policies, gives you both agility and reliability.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

19. You want to trigger Dataflow whenever a new file arrives in GCS. How would you build this event-driven pipeline?

Start with eventing: enable GCS object notifications to publish OBJECT_FINALIZE events to a Pub/Sub topic. This ensures you only react once the upload completes (avoid partial files). Use a small Cloud Function or Cloud Run service subscribed to that topic as the lightweight front door - it receives the event, validates bucket/prefix/filename against your expected contract, and normalizes the payload (include bucket, name, generation, size, eventTime) before taking action.

Decide how Dataflow should run: for per-file processing that's continuous/streaming, deploy a long-running Dataflow streaming pipeline that subscribes to that same Pub/Sub topic and processes each event in-line. For batch-style processing (one Dataflow job per file or per manifest), have the Cloud Function trigger a Dataflow Flex Template/Template v2 job via the Dataflow REST API or gcloud with the file path as a parameter. Flex Templates give you fast templated startups and parameter-driven behavior without rebuilding images constantly.

Make the pipeline idempotent and safe: include the GCS object generation or a deterministic run-id in the event and use it as a deduplication key. Keep a tiny idempotency store (Firestore or BigQuery control table) where triggers record attempted processing; the Dataflow job first checks this store and skips or aborts if the file was already processed. In the processing itself, write outputs to a staging prefix (or use transactional table formats like Hudi/Iceberg) and only atomically promote outputs (rename/copy + write _SUCCESS/manifest) after all checks pass.

Operational hardening: give the Cloud Function and Dataflow service accounts least privilege (Pub/Sub subscribe, Dataflow job create, GCS read/write for specific prefixes). Add validation in the trigger (file stability check if producers are flaky), emit structured logs and metrics (event arrival, validation failures, Dataflow launch latency), and enable Cloud Monitoring alerts for failed launches, DLQ counts, or high retry rates. Also version your Dataflow templates so you can roll back quickly if a new template misbehaves.

Finally, plan for scale and testing: if you expect bursts, use batching/aggregation (group small events for a single Dataflow job) or let Dataflow streaming scale out. Run chaos tests (simulate duplicate events, partial uploads, Storm of events) to ensure dedup and idempotency work. Document the contract for producers (expected prefixes, manifest behavior) so the whole event-driven system remains robust and predictable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

20. Some GCS event notifications are failing. How would you design error handling and retries?

Assume failures are normal and design a resilient pipeline: first, rely on Pub/Sub delivery guarantees. Use a Pub/Sub subscription with appropriate ack deadlines and retry policy; push-mode subscriptions should ack only after Cloud Function validation succeeds. If the function fails transiently, Pub/Sub will redeliver; if failures persist, configure a dead-letter Pub/Sub topic where events that exceed max delivery attempts land for offline triage.

In the Cloud Function (or Cloud Run) handler, validate events immediately and fail fast for transient issues (e.g., network hiccup) while moving permanent or malformed events to a DLQ. For example: if the event points to a missing or zero-byte object, push to a "quarantine" topic and create a diagnostic record (original event, producer id, timestamps) in BigQuery or Firestore. This preserves the raw event for manual inspection and prevents the system from looping on bad data.

For retries on Dataflow launch errors or downstream failures, implement exponential backoff with a capped retry count in the trigger service. If the Dataflow job creation fails repeatedly, push the trigger payload into an "orchestration retry" topic with a backoff scheduler or use Workflows/Cloud Tasks with delayed retries so you don't hammer the Dataflow API. Also keep a persistent control table recording attempts, last error and retry schedule - this enables automated retries while preventing runaway retries.

Handle duplicate and out-of-order events explicitly: use the object generation and metageneration fields from GCS as uniqueness and freshness checks. Before processing, consult your idempotency store (Firestore/BigQuery) to see if that bucket/name/generation was already processed; if yes, ACK and skip. For out-of-order (late) files, your pipeline should support watermarking or a late-data window, or else route late arrivals to a backfill queue for controlled processing.

Finally, observe and alert: emit metrics for delivery failures, DLQ rates, and retry counts; set alerts on thresholds so engineers investigate before backlog grows. Create runbooks that list common failure modes (auth errors, missing object, transient API limits) and the automated remediation steps the system already took. With Pub/Sub DLQs, explicit quarantine topics, controlled backoff retries, and idempotent processing, GCS notification failures become manageable rather than catastrophic.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

21. A business needs near real-time updates in GCS from Pub/Sub messages. How would you design the pipeline?

I'd use Pub/Sub as the durable ingestion layer and run a streaming Dataflow job as the primary mover to write batches to GCS. The Dataflow pipeline subscribes to the Pub/Sub topic, parses and validates messages, and groups them by a small, configurable window (for example 30s–2min depending on SLA) so we produce compact files instead of tiny objects. Windowing gives you a good compromise between latency and file-size efficiency while keeping downstream ETL simple.

Next, I'd design the output format and partitioning for downstream consumers: write compressed Parquet (or Avro if you need row semantics) into a Hive-style partition layout like gs://bucket/stream/<topic>/date=YYYY-MM-DD/hour=HH/. Include metadata (run id, watermark, message count) and write manifests or _SUCCESS markers so downstream jobs can safely discover completed files. Use deterministic file names that include a window ID and a content hash to make processing idempotent and easy to debug.

Operational robustness is key, so make the pipeline idempotent and handle duplicates. Use the message publish timestamp or an event id as a deduplication key (store recent keys in a small state store or rely on Dataflow's dedup features). Route malformed/poison messages to a dead-letter Pub/Sub topic with full context and alerts. Monitor end-to-end metrics: lag from publish to file finalization, file sizes, DLQ growth, and Dataflow worker health so you can detect backpressure early.

For scale and cost control, enable autoscaling on Dataflow and use batching or backpressure controls to avoid creating too many small files under load. If some consumers require sub-second updates, add a separate low-latency path (for example publish a thin event to a lightweight cache or Bigtable) while keeping the GCS path for durable analytics. This hybrid pattern preserves near-real-time requirements without sacrificing storage efficiency.

Finally, secure and automate the flow: give the pipeline least-privilege service accounts, enforce CMEK/VPC-SC if needed, and trigger downstream ETL (Dataproc jobs or Cloud Composer DAGs) after file finalization. Document the SLA (expected end-to-end latency) and run chaos tests (duplicate events, worker restarts) so the team is confident the "near-real-time to GCS" contract holds in production.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

22. A daily batch job loads millions of records into GCS. How would you explain when batch is better than streaming?

Batch is the right choice when latency requirements are lax (minutes to hours) and you can amortize overhead across many records - if you load millions of records once a day, a single well-optimized batch job will be far more cost- and resource-efficient than keeping a streaming pipeline running 24/7. Batch lets you coalesce work, compact outputs into large files, and avoid the small-file and coordination headaches streaming can introduce at scale.

Batch also simplifies correctness and idempotency: you can validate a complete dataset with manifests and checksums before promoting it to curated storage, making retries and reconciliation straightforward. For complex joins, heavy aggregations, or transformations that benefit from global windowing over a full day of data, batch engines (Dataproc/Spark or Dataflow batch) often perform simpler and faster than trying to approximate the same semantics in streaming with watermarking and state management.

Cost predictability is another strong argument: batch runs only when needed, so you pay for compute during the job instead of for continuously-provisioned streaming resources. For predictable daily workloads, ephemeral clusters and scheduled batch jobs let you right-size machines and use preemptible workers to lower cost without complicating the architecture with streaming autoscaling dynamics.

That said, streaming wins when business needs require low-latency reactions - monitoring, fraud detection, alerts, or near-real-time dashboards. If those use cases exist alongside daily analytics, consider a hybrid approach: a lightweight streaming path for immediate alerts and a batch pipeline for full, canonical loads into GCS and downstream warehouses. This keeps both latency-critical flows and efficient bulk processing happy.

So I'd pick batch for the millions-of-records-per-day use-case when latency tolerance, cost, and simplicity favor scheduled processing; choose streaming only if the business needs continuous, low-latency updates that can't wait for the daily window.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

23. Before loading data from GCS to BigQuery, you must validate schema consistency. How would you implement this?

I'd treat schema validation as a pre-load gate in the orchestration DAG so bad files never land in production tables. The DAG task would first discover the candidate files (manifest or prefix), then infer or read the incoming schema - for Parquet/Avro you can read the file-level schema from the file footer/metadata; for CSV/JSON you'd run a lightweight schema-detector on a sample or require an accompanying schema file (Avro/JSON Schema) that producers must provide. That gives you a concrete "incoming schema" to compare with the expected target schema.

Next, compare the incoming schema programmatically against the BigQuery target schema (or a canonical schema stored in Data Catalog / a schema registry). The comparison checks column names, data types with compatible promotion rules (e.g., INT→FLOAT allowed if agreed), nullability, required fields, and partition/partition-col presence. If the pipeline supports additive, safe changes (nullable columns added), allow them automatically; for breaking changes (type narrowing, renames, deletions) fail the run and emit a migration ticket so a controlled change can be applied.

If the schemas are compatible but the data needs simple normalization (timestamp formats, field renames, string→numeric casting), run a deterministic transform step that coerces fields into BigQuery-compatible types and writes to a staging location (Parquet preferred) before the load. If coercion would drop rows, capture those rows to a dead-letter path with the error reason and row sample so analysts can triage. For large or ambiguous mismatches, reject and notify producers with a clear contract violation report (expected vs observed schema, offending files).

Make the whole check auditable: record validation results, file checksums, sample row counts and column-level stats in a control table (BigQuery or Firestore) so you can prove what was validated and when. Integrate this into CI/CD by running schema compatibility checks whenever a producer changes schema - automated tests can catch drift before files ever arrive in GCS. Finally, add monitoring and alerts for schema drift metrics (new fields, type changes, failure rates) and a documented process for safe schema migrations so teams can evolve without breaking consumers.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

24. A file upload to GCS is incomplete or corrupted. How would you detect and handle it in your pipeline?

Design the ingestion contract so uploads are finalized atomically and observable: producers should either upload to a staging prefix (e.g., .../_tmp/...) and then move/rename to the final prefix once complete, or provide a manifest that lists all objects and their checksums. Use GCS OBJECT_FINALIZE events (not OBJECT_METADATA_UPDATE) to trigger processing, and in your trigger verify object metadata (size, crc32c/md5Hash, generation) before accepting the file for ETL - those fields let you detect zero-byte, truncated or tampered files quickly.

Always validate checksums as part of the pre-flight step. For files uploaded via client libraries or Storage Transfer Service, GCS stores crc32c and md5Hash (when available) - compare those with producer-supplied checksums in the manifest. For streaming writes, use resumable uploads and write-ahead metadata (sequence numbers) so incomplete sessions are detectable. If a checksum mismatch or zero-size is detected, move the object to a quarantine folder (e.g., gs://bucket/quarantine/<run>/) and push a structured error event to a DLQ (Pub/Sub or error topic) that includes the file path, expected checksum, observed checksum and producer id.

Implement automated remediation: small transient issues trigger a retry policy (re-run transfer or request producer to re-upload) with exponential backoff; repeated failures after N attempts escalate to on-call with the quarantined object preserved for investigation. For partial uploads caused by network blips, Storage Transfer Service or resumable uploads usually fix the problem - prefer those transfer methods when possible. Always avoid processing files that lack a _SUCCESS/manifest confirmation or fail checksum verification.

For downstream robustness, make your ETL idempotent and able to replay: keep the raw failed objects (with metadata) and allow reprocessing after a successful re-upload. Capture diagnostic artifacts (sample rows, schema inference failures) in the error record so debugging is fast. Finally, monitor upload health metrics (failed uploads, checksum mismatch rate, average object size) and alert on anomalies - proactive visibility plus quarantine+DLQ+retries turns incomplete/corrupt uploads from silent data loss into manageable incidents.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

25. A Spark job on Dataproc is failing due to excessive small files in GCS. How would you solve this?

First, stop the source of tiny files - if possible change the producer to write larger files (batch outputs instead of per-record writes, larger sink buffers, or bigger window sizes). Preventing tiny files is far cheaper than fixing them later. If the producer is a streaming sink, increase its window/batch interval or add batching at the gateway so fewer, bigger objects are created.

Second, perform a targeted compaction step in Spark on Dataproc. Run a job that reads the small files for a partition (or a set of partitions), repartition(n) or coalesce(n) to an appropriate number of output files that match your target size, and write compressed Parquet/ORC back to the same partition path (writing to a staging path and atomically swapping on success). Aim for tens-to-hundreds of MB per file depending on downstream read patterns. Use spark.read.option("mergeSchema", "false")... and df.repartition(...) carefully so you don't trigger unnecessary shuffles.

Third, adopt a table format that manages small-file churn: Hudi / Iceberg / Delta on top of GCS. These formats provide built-in compaction/cleaner operations and incremental reads which reduce the operational burden of manual compaction. They also make upserts and incremental ingestion safer so you don't create new tiny files for each micro-batch indefinitely.

Fourth, tune Spark read/write settings to be friendlier to many-file inputs while you transition: increase spark.sql.files.maxPartitionBytes so Spark groups many small files into fewer input partitions, and adjust spark.sql.shuffle.partitions to a sensible value for your cluster size to avoid creating too many small shuffle tasks. Also enable vectorized Parquet readers and tune executor memory/cores so per-file overhead is smaller.

Finally, automate and monitor: schedule regular compaction after ingestion (daily/weekly depending on volume), add metrics for file count per partition and average file size, and alert when small-file ratios rise. Tag jobs that are allowed to create tiny files and consider batching or periodic compaction for them. With producer-side batching + format-level compaction + one-time Spark compactions you'll remove the small-file problem and stabilize job performance.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

26. Your Dataflow pipeline writes thousands of tiny files to GCS. What strategies would you apply to compact them?

First, fix the sink behavior: change windowing/triggering so writes are coarser-grained. Use Beam/Dataflow windowing with reasonable durations (e.g., 1–5 minutes or longer depending on latency needs) and set triggers that reduce finalization frequency. That alone often collapses many tiny files into a handful of larger ones per window.

Second, control sharding and file-size targets at the sink. Use FileIO.write() or the Parquet/Avro IO with options like withNumShards(), withMaxRecordsPerFile() or withMaxBytesPerFile() where available, so the runner produces fewer shards and targets a desired bytes-per-file. If using GroupIntoBatches or GroupByKey before writing, you can aggregate records into larger bundles and then emit one file per bundle - this trades off some latency for much better storage efficiency.

Third, add an explicit downstream compaction step. If you cannot change the live streaming sink (because of other consumers or low-latency needs), run periodic compaction jobs: a batch Dataflow job or a Dataproc Spark job that reads the recent small files for a partition/window, merges them, and writes larger Parquet files to a compacted prefix, then atomically replaces the old files or writes manifests. For streaming-first architectures a scheduled compactor is a common hybrid approach.

Fourth, consider table formats or write-optimizing layers for streaming: if you need upserts or transactional semantics, use Hudi/Iceberg on top of GCS with a compaction/cleaner strategy. These systems can be driven from Dataflow or periodically from Dataproc to perform merge/compaction tasks and expose efficient snapshots for downstream consumers.

Finally, monitor and iterate: emit metrics for file-count-per-minute, average file size, and compaction backlog; set alerts when tiny-file rates spike. If you must use GCS compose to combine small objects, do so sparingly and be aware of limits - prefer producing correctly-sized files at source or via compaction jobs. Balancing window size, shard count, and a scheduled compaction process (or switching to a table format) will eliminate the tiny-file issue while preserving your latency and correctness requirements.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. You need raw files kept for 7 days and then automatically archived. How would you configure GCS lifecycle rules?

I'd configure an object lifecycle management policy on the raw bucket. The policy would have a rule that applies to all objects (or just the raw/ prefix if you're using a shared bucket) with the condition age > 7 days. The action for that rule would be to SetStorageClass → move the files into a cheaper archival tier like Nearline, Coldline, or Archive depending on access requirements. For example, if analysts occasionally reprocess the raw data, Nearline (30-day minimum) is a good fit; if the files are rarely touched after 7 days, Coldline or Archive would save more.

The steps would look like:

1. Create the raw bucket with versioning disabled (since we don't need older generations).
2. Define a lifecycle policy JSON like:

```
{
 "rule": [
  {
   "action": { "type": "SetStorageClass", "storageClass": "COLDLINE" },
   "condition": { "age": 7 }
  }
 ]
}
```

3. Apply it to the bucket with gsutil lifecycle set.

This ensures files remain in standard storage for 7 days (fast access for ETL or validation), then automatically move to the archival tier with no manual work. I'd also set bucket labels and monitoring so the team knows how much is in active vs archived tiers. That way raw is still recoverable if needed but cost-optimized after the 7-day hot window.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

28. Temporary staging data in GCS must be deleted automatically after ETL runs. How would you set this up?

For ephemeral staging, the simplest option is to apply a lifecycle delete rule to the staging/ prefix. Configure a lifecycle policy with a condition like age > 1 or even createdBefore if you want stronger guarantees. For example:

```
{
  "rule": [
    {
      "action": { "type": "Delete" },
      "condition": { "age": 1 }
    }
  ]
}
```

This deletes any object older than one day in the staging/ prefix, ensuring temporary files don't accumulate and inflate storage costs.

If you need tighter control - for example, delete immediately after a successful ETL - you can also make the deletion a Composer/Airflow DAG task (GCSToGCSOperator for move + delete, or GCSDeleteObjectsOperator for cleanup). That ensures you only clear staging after your _SUCCESS marker is written and validated. For safety, I'd use this explicit delete in the DAG for "job-local" staging, and use lifecycle rules as a backstop so orphaned files don't stick around if the DAG fails.

Finally, I'd isolate staging in its own bucket or prefix with minimal retention so lifecycle policies don't accidentally delete production data. Combine that with IAM (only ETL service accounts can write there) and monitoring (alerts on staging bucket growth) so temporary files are both controlled and cost-safe.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

29. You need to migrate a Hadoop data lake into GCS. How would you plan the migration?

Start with a thorough inventory and risk assessment. Catalogue data volumes, file formats, partitioning, number of files, Hive/Metastore schemas, custom SerDes/UDFs, owner teams, SLAs, and network topology. Measure total bytes, number of objects (small-file problem), and peak transfer throughput needs - this drives whether you use distcp/Storage Transfer Service, Transfer Appliance, or a phased, incremental plan. Also capture dependencies: which jobs read which paths, which downstream consumers must be paused or dual-run during cutover.

Design the target layout and metadata mapping up front. Decide bucket(s) and prefixes for raw/curated/analytics, choose partitioning conventions (Hive-style key=value/), and pick canonical formats (Parquet/ORC) for curated data. Export and test migrating the Hive Metastore (or plan a Dataproc Metastore migration) so table definitions, partitions, and location URIs point to gs://... after migration. Consider adopting a table format (Hudi/Iceberg) during migration if you need upserts/time-travel after move - it's a one-time ops cost that pays off later.

Plan the transfer strategy in phases: pilot (small datasets + representative schemas), bulk copy with validation, incremental sync for near-zero-downtime cutover, then final cutover. For transfer mechanics use Hadoop distcp to copy HDFS→GCS (via GCS connector) or Storage Transfer Service/gsutil for object-level copy; for very large petabyte data consider Transfer Appliance or bandwidth reservation (Interconnect/Partner). Preserve checksums, object metadata, and timestamps where possible; run automated validations (file counts, checksums, row counts) and smoke-tests (run a few representative queries/jobs against GCS data to validate parity).

Coordinate application and job migration: update ETL and query jobs to use gs:// URIs, adjust configs (shuffle/local disk vs remote), retune Spark/Tuning parameters for cloud VMs, and stage tests in a parallel environment. Use ephemeral Dataproc clusters for conversion/compaction (convert CSV→Parquet, compact tiny files, register partitions in metastore). Keep a dual-running window where producers write to both HDFS and GCS or sync changes until validation and stakeholder sign-off.

Finally, operationalize and rollback planning: automate migration steps in CI/CD or runbooks, capture lineage & ownership in Data Catalog, enforce IAM and encryption (CMEK) on target buckets, and put lifecycle rules in place. After cutover, decommission HDFS carefully (retain a frozen snapshot for a retention period as a rollback option). Monitor costs, query performance, and run post-migration tuning (file sizes, partitioning, executor sizing). A planned, measured, test-first migration with clear validation gates makes the move low-risk and repeatable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

30. Your ML pipeline requires storing feature sets in GCS before training in Vertex AI. How would you design this workflow?

Design with reproducibility and low-latency serving in mind. Compute features (batch or streaming) using Dataflow/Dataproc depending on workload, and write canonical, versioned feature snapshots to GCS in a query- and ML-friendly format. For tabular features prefer columnar Parquet for offline training (good for Spark/Pandas) or TFRecord/TFExample for TensorFlow pipelines - pick the format that the training framework reads most efficiently. Partition by date/run and include metadata (run-id, schema version, provenance JSON) next to the snapshot so you can reproduce training datasets exactly.

Enforce schema and versioning. Maintain a feature schema registry (in Data Catalog or a simple metadata table) that records feature names, types, nullability, and transformations. Each feature snapshot folder should include a manifest and a schema file so training jobs and audits can verify compatibility. Make snapshots immutable - never overwrite a published training snapshot; write a new version for every retrain so experiments are reproducible and rollback is easy.

Wire training to these snapshots predictably. Vertex AI training jobs can read directly from GCS (Parquet/TFRecord) or BigQuery; choose GCS when you want file-based input and predictable sharding, or load into BigQuery when you want SQL-based feature assembly. Before launching Vertex AI training, run a pre-check DAG step that validates row counts, label balance, and feature distributions against historical baselines to catch drift early. Persist the exact GCS URIs used for training in your model registry or experiment metadata.

Support online/serving use-cases by deciding how features flow from batch snapshots into an online store. If low-latency serving is required, export features into Vertex Feature Store (or Bigtable/Redis/BigQuery for lookup patterns) after computing them, keeping the same schema and entity keys. Use the same ETL code for both offline snapshot writing and online ingestion to ensure parity between training and serving features (the “same code in dev and prod” principle).

Automate and secure the flow. Orchestrate feature computation, validation, snapshotting, and Vertex AI training with Airflow/Composer so runs are auditable and retriable; record lineage and metadata at each step. Use least-privilege service accounts for GCS and Vertex AI, enable CMEK/VPC-SC if needed, and add monitoring for job failures, data drift metrics, and storage usage. With immutable, versioned snapshots, schema governance, and automated validation, your Vertex AI training runs will be reproducible, debuggable, and safe to promote into production.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar